ARTIFICIAL INTELLIGENCE

1.Write the python program to solve 8-Puzzle problem

Python code:

```
from collections import deque


s=input("Enter puzzle: ")

q=deque([(s,[])]) v={s}


while q:

 x,p=q.popleft() if

x=="123456780":

print("Solution:")

for a in p+[x]:

   print(a[:3]);print(a[3:6]);print(a[6:]);print()

break i=x.index("0") for j in [i-3,i+3,i-

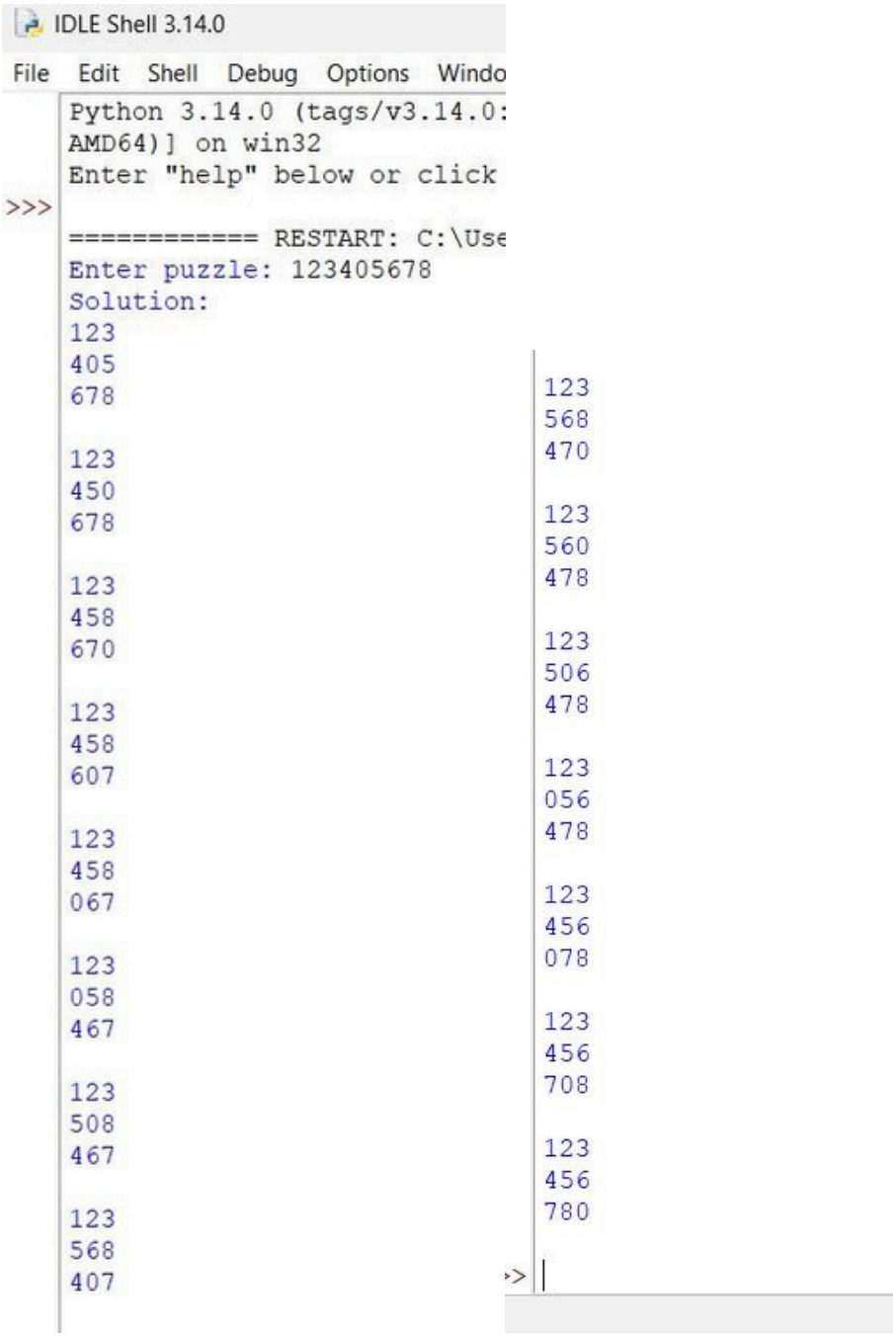
1,i+1]: if 0<=j<9 and abs(i%3-j%3)<=1:

   a=list(x);a[i],a[j]=a[j],a[i];a=".join(a)

if a not in v:v.add(a);q.append((a,p+[x]))

else: print("No solution")
```

Output:



2.Write the pythonprogram to solve 8-Queen problem

Python code:

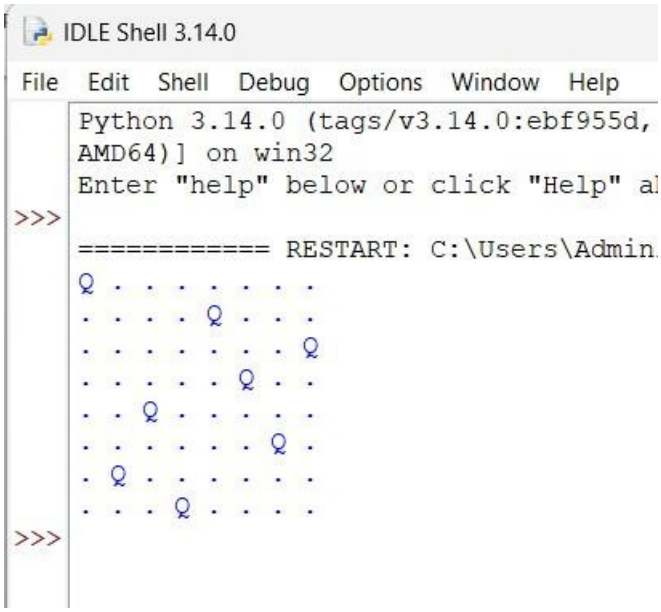
def solve(r, q): ifr

== 8: foriin

range(8):

```
            print(" ".join("Q" if q[i] == j else "." for j in range(8)))
return True     for c in range(8):
        if c not in q and all(abs(c-x) != r-i for i,x in enumerate(q)):
            if solve(r+1, q+[c]):
                return True
return False
```

solve(0, []) **Output:**



3.Write the python program for Water Jug Problem

Python code:

a=0 b =

0print(a,

b)

while a!=2: if a

==0: a =4

elif b==3: b=

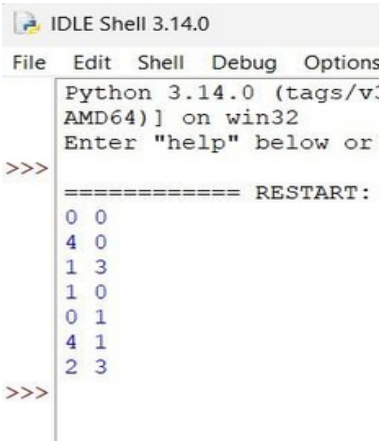
0 else: x=

min(a,3-b) a =

a-x b=b+x

print(a, b)

Output:



4.Write the python program for Missionaries Cannibal problem

Python code:

from collectionsimport deque

```
q = deque([((3,3,1), [])]) v =

{(3,3,1)} moves =

[(1,0),(2,0),(0,1),(0,2),(1,1)]



while q:

   (m,c,b), p = q.popleft()

if (m,c,b) == (0,0,0):

for x in p + [(m,c,b)]:

          print(x)

break for dm,dcin

moves:

       nm=m-dmifbelsem+dm

nc = c-dcifbelsec+dc

       nb=1-b          if0<=nm<=3and0<=nc<=3:              if

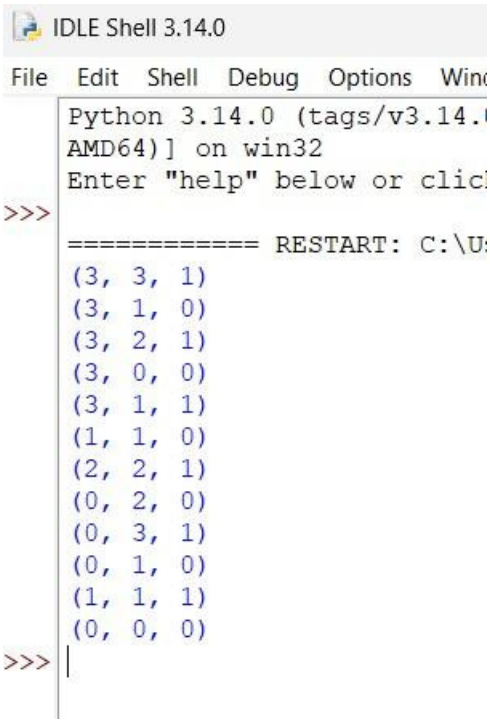
(nm==0 or nm>=nc) and (3-nm==0 or 3-nm>=3-nc):

             s=(nm,nc,nb)

if snotinv:

                v.add(s);q.append((s,p+[(m,c,b)]))
```

Output:



5.Write the python to implement Travelling Salesman Problem

Python code:

```
from itertoolsimport permutations


d=[[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]

best=999 path=None


for p in permutations(range(1,4)):

   r=(0,)+p+(0,)     cost=sum(d[r[i]][r[i+1]]

for i in range(4))     if cost<best:

best=cost         path=r
```

print("Path:",path) print("Minimum

cost:",best) **Output:**

